

Software Performance and Scaling II

From Algorithmic Complexity to Hardware Acceleration

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

In this second part on software performance, we move beyond basic profiling to explore advanced optimisation strategies. We will examine the trade-offs between language efficiency and algorithmic complexity, explore concurrency through processes and threads, and look at how to leverage modern hardware parallelism (SIMD, GPUs, and TPUs).

- 1 Language Efficiency vs. Algorithmic Complexity
- 2 Concurrency: Processes vs. Threads
- 3 Hardware Parallelism: SIMD and Vectorisation
- 4 Hardware Acceleration: GPUs and TPUs
- 5 The Limits of Scaling
- 6 Summary

Language Efficiency vs. Algorithmic Complexity

Language Efficiency vs. Algorithmic Complexity

The Baseline: Calculating Primes in Python

To compare performance, we start with a simple algorithm to calculate primes up to 100,000.

Listing 1: primes.py

- We iterate through numbers and try to divide them by previously found primes.
- **Result:** Takes approximately **5 seconds**.
- Uses 'append' and list iteration.

```
1 primes = []
2 for test in range(2,100000):
3     is_prime = True
4     for p in primes:
5         if test % p == 0:
6             is_prime = False
7             break
8     if is_prime:
9         primes.append(test)
```

Pitfall: Premature Optimisation

A common mistake is assuming native Python operations (like 'append') are the bottleneck and trying to write "C-style" code in Python.

The "Optimised" Attempt

We pre-allocate the list (`[0 for i in ...]`) to avoid appending, and use index-based iteration instead of `for p in primes`.

The Result

- **Time:** Increases to nearly **8 seconds**.
- **Lesson:** Native Python primitives are highly optimised in C. Writing "C-style" logic in Python often adds overhead. This is a classic example of premature optimisation.

The Language Factor: Switching to C

If raw speed is required, we can switch to a lower-level language.

Listing 2: primes.c

```
// Same logic, rewritten in C
for (int index = 0; index < primecount;
     index++) {
    if (test % primes[index] == 0) {
        is_prime = false;
        break;
    }
}
```

- **Unoptimised C:** 0.17 seconds.
- **Optimised C (-O):** 0.128 seconds.
- **Improvement:** $\approx 50\times$ faster than Python.

Compiler Optimisation

Turning on the optimiser allows the compiler to reorganize code to better fit CPU pipelines and superscalar execution.

Hybrid Approaches: Python + C

You do not need to rewrite your entire codebase in C.

- We can compile the C code as a **Shared Library** (‘.so’) using flags ‘-c -fPIC’ and ‘-shared’.
- Python’s ‘ctypes’ library allows us to call these C functions directly.

Performance

Time: 0.145 seconds.

- Almost identical to native C speed.
- Allows keeping high-level logic in Python while offloading compute-heavy tasks to C.

The Algorithm Factor: Complexity Matters Most

Before optimising code or switching languages, we must question the algorithm itself.

The Sieve of Eratosthenes

Instead of trial division, we assume all numbers are prime and iteratively "cross out" multiples of known primes.

Listing 3: sieve.py

```
# ... initialisation ...  
while p <= sqrt_n:  
    if prime[p]:  
        for i in range(p * 2, n + 1, p):  
            prime[i] = False  
    p += 1
```

- **Time:** 0.03 seconds.
- **Result:** This Python code is $4\times$ faster than the optimised C code using the inefficient algorithm.

The Algorithm Factor: Complexity Matters Most

The Sieve of Eratosthenes

Instead of trial division, we assume all numbers are prime and iteratively "cross out" multiples of known primes.

Listing 4: sieve.py

```
# ... initialisation ...  
while p <= sqrt_n:  
    if prime[p]:  
        for i in range(p * 2, n + 1, p):  
            prime[i] = False  
    p += 1
```

- **Time:** 0.03 seconds.
- **Result:** This Python code is $4\times$ faster than the optimised C code using the inefficient algorithm.

Key Takeaway

$O(N)$ improvements beat constant factor speedups. Algorithm first, efficiency second.

Concurrency: Processes vs. Threads

Concurrency: Processes vs. Threads

Utilising Multiple Cores

Modern CPUs have multiple cores (e.g., 4 to 64+). Standard Python scripts run on a single core.

- **Single Core:** The OS switches between processes rapidly to give the illusion of parallelism.
- **Multi-Core:** We can actually run code simultaneously.

Option 1: Multiple Processes

A process has its own virtual memory space.

- **Pros:** Strong isolation; if one crashes, others survive. Bypasses Python's GIL.
- **Cons:** Memory is not shared. Data must be copied (pickled) between processes.

Python Multiprocessing

We can use the 'multiprocessing' library to spawn worker processes.

Listing 5: process.py snippet

```
1 from multiprocessing import Process
2
3 def hold(sec):
4     time.sleep(sec) # Simulate work
5
6 if __name__ == "__main__":
7     processes = []
8     for seconds in [2, 3, 4]:
9         proc = Process(target=hold, args=(seconds,))
10        processes.append(proc)
11        proc.start()
12
13    for proc in processes:
14        proc.join() # Wait for completion
```

Result: Running three tasks (2s, 3s, 4s) takes ≈ 4 seconds total, proving parallel execution.

Modern Concurrency: `concurrent.futures`

Managing raw processes and queues can be error-prone. Modern Python provides a higher-level abstraction.

```
1 from concurrent.futures import ProcessPoolExecutor
2 def hold(sec):
3     time.sleep(sec)
4     return f"Done in {sec}s"
5 # Automatically manages the pool of workers
6 with ProcessPoolExecutor() as executor:
7     results = list(executor.map(hold, [2, 3, 4]))
8 print(results)
```

Benefits

- Handles worker lifecycle (start/join) automatically.
- Easy retrieval of return values (Futures).
- Switch between `ProcessPoolExecutor` and `ThreadPoolExecutor` with one line.

Inter-Process Communication

To get results back from a process, we cannot simply 'return' a value because the memory spaces are separate.

- We must use a **Queue** or Pipe.
- This introduces overhead: data is serialized, sent to the main process, and deserialized.

Efficiency Note

Spawning processes and copying data is expensive. This model works best for "long-lived" tasks that crunch data for a while and return a concise result.

Option 2: Threads

Concepts

Threads of execution exist **within the same process memory space**.

- **Pros:** Fast data sharing (no copying required).
- **Cons:** Danger of interference (Race Conditions).

Race Conditions

When two threads modify the same data structure simultaneously without coordination, data corruption occurs.

The Critical Section Problem

Example: Inserting into a Linked List.

- 1 Thread 1 reads `prev->next`.
- 2 Thread 2 reads `prev->next` (same location).
- 3 Thread 1 updates pointers.
- 4 Thread 2 updates pointers, overwriting Thread 1's work.
- 5 **Result:** An element is lost (memory leak or crash).

Solution: Locks (Mutex)

We must define **Critical Sections** where only one thread can enter at a time.

- 'Lock.acquire()' → Critical Section → 'Lock.release()'

The Cost of Locking

Locks introduce serialisation and potential performance cliffs.

Case Study: Windows NTFS Lock

A 64-core system was observed freezing periodically.

- **Diagnosis:** One core held a lock on the NTFS file system.
- **Impact:** All other 63 cores were blocked waiting for that lock.
- **Result:** Performance dropped to effectively zero despite massive hardware.

Hardware Parallelism: SIMD and Vectorisation

Hardware Parallelism: SIMD and Vectorisation

Beyond Multi-Core: Hyper-Threading

Hyper-Threading

Allows a single physical core to execute two threads by utilising idle time (e.g., waiting for memory fetches).

- Typically yields $\approx 1.5\times$ speedup, not $2\times$.

Instruction Level Parallelism

Standard CPUs operate as SISD (Single Instruction, Single Data). To go faster, we can use SIMD (Single Instruction, Multiple Data).

SIMD and AVX-512

Modern CPUs have wide registers (e.g., 512-bit).

- We can load multiple pieces of data (e.g., 16 floats) into one register.
- A single instruction executes the operation on all 16 values simultaneously.

Performance Potential

With AVX-512, a Xeon processor can execute **64 single-precision floating point operations per cycle per core**.

Accessing SIMD in Python

You cannot write SIMD instructions directly in pure Python.

- **NumPy**: Delegates array operations to optimised C backends (BLAS/LAPACK).
- **Numba**: A JIT compiler that translates Python functions to optimised machine code (LLVM), often automatically vectorising loops (SIMD) to reach C-like speeds.

Hardware Acceleration: GPUs and TPUs

Hardware Acceleration: GPUs and TPUs

Graphics Processing Units (GPUs)

For massive parallelism (e.g., matrix multiplication), CPUs are outperformed by GPUs.

Architecture

- **CPU:** Few powerful cores, complex caching/branch prediction.
- **GPU:** Thousands of simpler cores (Stream Processors).

Scale

- Example (RTX 2080 Ti): >4000 cores.
- Performance: 13 Teraflops (single precision).
- Ideal for SIMD-like workloads.

Programming GPUs with PyCUDA

We can write C-like kernels to run on the GPU from Python.

Listing 6: cuda.py snippet

```
1 mod = comp.SourceModule("""
2 __global__ void multiply_them(float *dest, float *a, float *b) {
3     const int i = threadIdx.x;
4     dest[i] = a[i] * b[i];
5 }
6 """)
7
8 # ... set up arrays with numpy ...
9 multiply_them(drv.Out(dest), drv.In(a), drv.In(b), block=(400,1,1))
```

This executes 400 multiplications in parallel on the GPU hardware.

Modern GPU Programming: PyTorch & JAX

Modern frameworks abstract away the C-kernels, allowing us to write Python code that runs on CUDA (NVIDIA) or MPS (Apple Silicon).

PyTorch (CUDA/MPS)

```
1 import torch
2 # Auto-select device
3 device = "cuda" if torch.cuda.is_available() \
4         else "mps" if torch.backends.mps.is_available() \
5         else "cpu"
6
7 x = torch.randn(1000, 1000, device=device)
8 y = x @ x # Matrix multiplication on GPU
```

JAX (XLA)

```
1 import jax.numpy as jnp
2 from jax import jit
3 # JIT compiles to XLA (GPU/TPU)
4 @jit
5 def matmul(a, b):
6     return jnp.dot(a, b)
7
8 x = jnp.ones((1000, 1000))
9 y = matmul(x, x) # Runs on accelerator
```

MLX (Apple Silicon)

```
1 import mlx.core as mx
2 # MLX is designed for Apple Silicon (Unified Memory)
3 a = mx.random.normal((1000, 1000))
4 b = mx.random.normal((1000, 1000))
5 c = a @ b # Operations run on GPU/NE automatically
6 mx.eval(c) # Lazy evaluation: computation happens here
```


Tensor Processing Units (TPUs)

For Machine Learning, we can go even further with domain-specific hardware.

Google TPUs

- Designed specifically for neural network inference and training.
- **Trade-off:** Lower precision (e.g., 16-bit float / bfloat16) for massive throughput.
- **High Interconnects:** Pods of 1024 TPUs can achieve >100 Petaflops.

The Limits of Scaling

The Limits of Scaling

Amdahl's Law

Despite massive hardware parallelism, we are limited by the serial portion of our code.

The Formula

The maximum speedup S is given by:

$$S = \frac{1}{(1 - p) + \frac{p}{s}}$$

Where p is the parallelizable fraction of the code, and s is the number of cores.

Implication

If only 95% of your code is parallelisable ($p = 0.95$), the maximum theoretical speedup is $20\times$, regardless of whether you have 100 or 10,000 cores.

Summary

Summary

Summary

Key Takeaways

- **Measure First:** Avoid premature optimisation.
- **Algorithms Win:** Better complexity ($O(N)$ vs $O(N^2)$) beats language efficiency.
- **Concurrency:** Processes allow isolation (good for Python); Threads share memory but risk race conditions.
- **Hardware:** Use NumPy to leverage CPU SIMD (AVX). Use GPUs/TPUs for massive data parallelism.
- **Reality Check:** Amdahl's Law limits the maximum speedup based on the serial portion of your code.